

Slide 01 / Introduction

Explain: Introduce ARRAKIS: Harvester Down - Desert Rescue, a C++17 and OpenGL 3.3 core real-time graphics project. The player flies an ornithopter to rescue 36 harvester workers before and during a sandworm attack. Eight seats require repeated pickup and delivery trips; only workers delivered to the cyan base count as secured.

Point out: Identify the presenter as Asique Ehetasamul Haque, roll 2107096, Year 4, Semester 1, Group B2, Department of CSE, KUET. The course is CSE 4102, Computer Graphics and Image Processing Laboratory. Connect the desert scene to the three central goals: recognizable procedural objects, mathematical motion, and meaningful player interaction.

Demonstrate: Use the opening gameplay image to identify the aircraft, dunes, and worm threat. Slides 3-4 supply enlarged harvester and safe-base views. Describe the task rather than waiting for a full mission during the introduction.

Implementation: The application combines reusable generated meshes, GLSL shaders, mission state, custom HUD, and spatial audio. Source responsibilities are expanded on Slide 2.

Accuracy: The project models and visual surface details are generated in code, not imported models or image textures. This statement does not include the externally loaded audio files. The deck contains exactly ten slides; the existing demonstration video is external, not embedded.

Slide 02 / Outline

Explain: Follow the presentation from object construction to curved geometry, coordinated motion, controls and cameras, rendering, feedback, and evaluation. The rescue loop is fly to an amber group, hover low and slowly, winch workers aboard, return to the cyan base, unload, and repeat while the worm closes in.

Point out: Link the source modules to that loop: `src/arrakis.cpp` owns meshes, object assembly, shaders, callbacks, and the render loop; `arrakis_game.h` owns mission and rescue rules; `arrakis_worm.h` owns the detailed worm; `arrakis_hud.h` owns instrumentation; `arrakis_audio.h` owns sound. Tests and CMake provide verification and build integration.

Demonstrate: Trace one frame through input, bounded mission updates, camera and audio updates, shadow depth rendering, procedural sky, lit opaque geometry, sorted transparent dust, HUD, and buffer swap. Distinguish CPU state updates from GPU vertex and fragment processing.

Implementation: Mesh positions, normals, UVs, and triangle indices are uploaded through VAO/VBO/EBO resources. Model, view, and projection matrices connect assembled objects to the camera, while the light-space matrix supports shadows.

Accuracy: The main scene renders directly to the default framebuffer. Tone mapping is inside the surface shader, not a separate bloom or post-processing pipeline. Later slides give the detailed evidence; this outline is not an extra feature slide.

Slide 03 / Objects I / Desert and machines

Explain: This first object contact sheet inventories the environment and industrial vehicles. The terrain is a sampled continuous dune height field, the aircraft is an articulated assembly, the harvester is a moving crawler, and the tanks and rocks establish recognizable desert infrastructure. Different materials make reused primitives read as different objects rather than identical blocks.

Point out: On the aircraft, identify the ellipsoidal fuselage and canopy, forward sensor probe, intake cowls, wing gimbals, eight tapered aerofoil blades, twin turbines and nozzles, emissive exhaust, tail boom, V-tail, and landing skids. On the harvester, identify the hopper and refinery decks, four track units, road wheels, circulating tread pads, cutter drum, side exit ramp, separator funnels, bridge viewports, lamps, stacks, and spice-colored vent rims.

Demonstrate: Read the labels against the actual close-ups. The three pressure tanks use a horizontal cylinder, overlapping rounded end heads, reinforcing bands, hatch, relief valve, inclined cradle supports, skid beams, and an emissive sight tube. Each of eight rock sites assembles four differently transformed eroded-rock pieces with sediment-like shader bands. The terrain supports all these objects, rather than serving as a flat backdrop.

Implementation: `src/arrakis.cpp` builds cube, cylinder, UV sphere, wing, terrain, and eroded-rock meshes, then `drawPart` applies parent-relative translation, rotation, and scale. Startup uses `createDuneTerrain(260,260,900)`: 68,121 vertices and 135,200 triangles. Cylinder caps have separate normals; rock triangles use faceted normals.

Accuracy: Contact-sheet close-ups are staged isolated views using the exact project draw routines and actual models, not independently remodeled illustrations or ordinary gameplay moments. Tank positions and sight-tube emission are constant; there is no player deployment or pulsing level simulation. The canopy and bridge glass are shiny opaque materials, and emissive lamps are not spotlights.

Slide 04 / Objects II / Rescue and sandworm

Explain: The second object slide connects the rescue markers and worker models to the sandworm threat. The large mouth view should reveal a real recessed cavity, not a flat dark circle. Separate recognizable objects from the effects that communicate activity and danger around them.

Point out: Identify the worm's ridged deformable body, thick irregular lip, inward-facing funnel, deeply recessed throat, and six concentric rows of curved pointed teeth. The row counts are 192, 184, 176, 168, 152, and 128, totaling 1,000 teeth. The mouth narrows toward its recessed closure, leaving the visible opening unobstructed. Four sand-colored mound shapes form a wake behind the approaching worm.

Demonstrate: Identify the cyan base's circular pad, luminous ring, cross markings, and four beacon poles. Contrast these safe-base cues with an amber evacuation ring and its pole-top beacon. A worker uses ellipsoidal torso, head and visor, cylindrical arms and legs, and a small cyan overhead marker. During active pickup, the targeted worker rises and a thin cyan winch cylinder joins the aircraft to the worker. Amber group markers disappear when no running or waiting crew remain in that group.

Implementation: `src/arrakis_worm.h` creates five indexed meshes: body, lip, funnel, throat, and teeth. The body uses 192 longitudinal rows and 96 angular slices; revolved mouth profiles use 160 slices. Only the flexible body is streamed when the pose changes, and both shadow and scene passes share the pose. `drawRescueWorld` in `src/arrakis.cpp` constructs the base, markers, workers, and winch; marker rings align to the local terrain normal.

Accuracy: These are generated project objects shown through the shared draw routines. Worker close-ups can be staged for inspection. Dust is transparent billboard geometry, whereas the wake mounds are solid sphere-based objects. Workers aboard or already safe are state records, not separately rendered seated passengers or a populated interior cockpit.

Slide 05 / Curved geometry / The equations behind the objects

Explain: Relate each equation to its visible object. With angles in radians, the raw dune height is $h_0 = 28 \sin(.009x + .004z) + 19 \cos(.004x - .012z) + 5(1 - \abs{\sin(.028x + .016z)})^2 + 1.8 \sin(.065x + .038z) - 4$. The first terms create broad dunes, the squared term creates ridges, and the last adds small undulations.

Point out: The actual terrain adds base leveling: $b = 1 - \text{smoothstep}(15, 25, \text{distance}((x, z), (55, 55)))$; $h = (1 - b)h_0 + b h_0(55, 55)$. An ellipsoid is $(a \sin(\theta) \cos(\phi), b \cos(\theta), c \sin(\theta) \sin(\phi))$; the source sphere has radius .5 before scaling. Cylinder sides are $(.5 \cos(\phi), y, .5 \sin(\phi))$, y in $[-.5, .5]$, with separate cap fans. The torus is $((1 + .025 \cos(\beta)) \cos(\alpha), .025 \sin(\beta), (1 + .025 \cos(\beta)) \sin(\alpha))$, matching the base and evacuation rings.

Demonstrate: Follow the worm surface $P(t, a) = C(t) + (R(t) + \text{grain}) * (\cos(a), -T_z \sin(a), T_y \sin(a))$, where T is the unit centerline tangent. Below $t = .4$, C is the cubic Bezier sum $(1-s)^3 P_0 + 3(1-s)^2 s P_1 + 3(1-s)s^2 P_2 + s^3 P_3$. Above the join, $C = \text{end} - (0, \text{span} \cos(\text{mid}), \text{span} \sin(\text{mid}))$, $\text{span} = 64(1-s) \text{sinc}(\text{tilt}(1-s)/2)$, $\text{mid} = \text{tilt}(1+s)/2$. The radius blends 13.9 plus a .70-amplitude, 32-cycle cubed-cosine ridge toward 15*aperture at the mouth.

Implementation: Each tooth centerline is $\text{radial} * (r - \text{reach} * u) + \text{sideways} * \text{curl} * u^2 + (0, \text{depth} + .65 \sin(\pi * u) - \text{drop} * u^2, 0)$. Sweep five-sided cross sections in its tangent frame with radius $\text{width} * (1-u)^{.85}$, then add a true apex. Source references are `rawDuneHeight/createSphere/createCylinder/createRing` in `arrakis.cpp` and `deformBody/makeTeeth` in `arrakis_worm.h`.

Accuracy: $\text{sinc}(q) = \sin(q)/q$ with its limiting value at zero. Curves are sampled triangle meshes, not exact analytic raster surfaces. Terrain normals use finite differences with epsilon .4; worm normals include bend and ridge derivatives. Increasing sampling density improves the approximation but does not change its mathematical definition. Distinguish a curved centerline from a straight cylinder merely rotated into place. The crater is a separate GPU displacement described on Slide 8.

Slide 06 / Complex motion / One changing rescue scene

Explain: Dynamic scenes combine continuous mathematical motion with discrete mission events. For flight, $v' = v + (v_{\text{Desired}} - v) * (1 - \exp(-k * dt))$, then $p' = p + v' * dt$; k is 5.5 normally or 12 while winching. Normal and boost targets are 28 and 48 units/second. Heading smoothing is analytically integrated; bank, pitch, altitude, and camera elevation use exponential damping rather than abrupt changes.

Point out: Wing angles are $34 * \sin(\text{phase})$ and $34 * \sin(\text{phase} + .75 * \pi)$ degrees, with mirrored signs, fixed sweep/dihedral, and $8 * \cos(\text{phase}) * \text{side}$ pitch twist. phase advances at 23 radians/second normally and 36 while boosting. The harvester follows $x = -40 + .62 * t + 15 * \sin(.045 * t)$, $z = -55 + 22 * \sin(.032 * t) + 9 * \sin(.071 * t)$; $\text{yaw} = \text{degrees}(\text{atan2}(-dx/dt, -dz/dt))$. Accumulated horizontal travel drives tread pads, wheel details, and the cutter. Its body follows dune height and local slope.

Demonstrate: Read the four-stage worm filmstrip as approach, rise, lunge/swallow, and retreat. Approach gap interpolates 220 to 18 units. Relative to breach, rise uses smoothstep(0,6), lunge smoothstep(6,12), and retreat smoothstep(13.5,18); the harvester sinks 20 units during seconds 6-12 and then disappears. Crew batches contain 1-4 workers, released at $6 + 7 * j$ seconds at the route's scheduled position. Workers run at 3.8 units/second; limb angles use $24 * \sin(11 * \text{time} + i)$ while running or $4 * \sin(2 * \text{time} + i)$ while waiting.

Implementation: `arrakis_game.h` handles release, pursuit, proximity losses, and rescue states. `arrakis_worm.h` uses the smoothed stages to drive height, reach, tilt, and aperture. Normal missions currently have a 150-second pursuit followed by at most 60 seconds of extraction, with earlier closure possible. Nearby low aircraft and exposed crew can be lost; remaining inside crew are lost at attack second 12.

Accuracy: `resetMission` currently uses $\max(150, 10 - (\text{wave} - 1) * 10)$, so ordinary waves do not progressively shorten. The report's older 210-to-150 progression is stale. This is authored kinematics and timed hazard logic, not a general physics or navigation engine.

Slide 07 / Game-like controls / Fly, inspect, rescue

Explain: The player can control translation, heading, height, boost, winching, mission state, and view selection. W/S provide forward/backward thrust; A/D strafe; Q/E raise/lower requested terrain-relative altitude. Mouse horizontal displacement and Left/Right arrows steer. Mouse vertical displacement and Up/Down arrows change chase-camera elevation, not aircraft pitch directly.

Point out: Space brakes to ten percent of the movement target, lowers requested hover altitude toward six units, disables boost, and enables pickup or unloading. Left Shift boosts while moving; the reserve drains at .22/second and refills at .14/second, with a low-reserve lock requiring Shift release. C cycles wide chase, close chase, and tactical overhead. Enter starts or advances after debrief; R retries; P pauses; M mutes; F toggles wireframe; F11 toggles fullscreen; Escape exits.

Demonstrate: Compare all three camera images. Wide chase uses distance 40, close chase distance 20 with an additional 2.2 vertical offset, and overhead uses 26 behind plus 100 above. For a rescue, approach within 12 horizontal units of a running or waiting worker, stay at or below 11 units of ground clearance and below speed 6, and hold Space for .42 seconds per person. Capacity is eight. Return within 14 horizontal units of base and meet the same low/slow conditions for a 1.2-second unload of the entire cabin. Only unloading increases the secured score.

Implementation: `arrakis_game.h` implements `setMouseHover`, `updateMission`, `flightCameraPitch`, and `flightCameraOffset`; `src/arrakis.cpp` supplies callbacks and `lookAt/perspective` matrices. Mouse steering has a normalized .12 deadzone. The camera shares the aircraft's heading exactly while its target and elevation are damped, and it stays above terrain. Aircraft altitude requests are clamped to 4-55 with extra obstacle clearance.

Accuracy: The close preset is external third-person, not a modeled cockpit. The cursor is window-position steering, not a captured free-look mouse. Invalid pickup conditions or a changed target reset progress; boarding alone does not save a worker from a subsequent aircraft loss.

Slide 08 / Rendering / What makes the scene convincing

Explain: Rendering adds readable form, depth, atmosphere, and activity without imported image textures. The pipeline uses a shadow depth pass followed by procedural sky, lit opaque objects, transparent dust, and HUD. A 2048-by-2048 shadow map follows the player; the scene samples a 5-by-5 PCF kernel with receiver-slope correction and bias to soften edges and reduce acne.

Point out: Materials combine warm ambient light, directional sunlight, Blinn-Phong specular, and emission. Sand uses wrapped diffuse $\max((N \cdot L + .35)/1.35, 0)$, plus distance-faded sine ripples, slope-based coloring, perturbed normals, and sparse quartz sparkle. Rocks use procedural grain and height-dependent strata. Emission brightens beacons, spice indicators, and exhaust without making them separate lights. The sky shader draws a gradient and sun disc/halo from a full-screen triangle.

Demonstrate: Compare shadowed and sunlit areas, close sand detail and distant fog, then identify the worm crater and several dust sources. Fog weight is $1 - \exp(-(distance * density)^2)$. Surface tone mapping is $\text{pow}(\text{lit}/(\text{lit} + .8), 1/1.8)$, followed by fog blending. Crater displacement is $-13 * \text{collapse} * (1 - \text{smoothstep}(5, 48, d))$, with $\text{collapse} = \text{smoothstep}(7, 14, \text{attackTime})$; both scene and shadow vertex shaders apply it.

Implementation: `src/arrakis.cpp` maintains 2,400 CPU-updated particles with lifetime, velocity, ground contact, respawn, and gust modulation. Four emission categories are ambient wind, crawler plume, worm wake/breach, and winch downwash. Sorted instance records provide position/size and color/alpha for camera-facing quads; one `glDrawElementsInstanced` call renders them with alpha blending and depth writes disabled. Fade-in/out and a procedural soft mask replace particle image textures. Four-sample MSAA is requested, VSync is enabled, and F exposes mesh wireframe.

Accuracy: The crater is GPU-only; CPU terrain queries and collision clearance do not acquire the depression, and precomputed terrain normals are not rebuilt for it. Fog is analytic, shadows cover a finite region, and dust is billboards rather than volumetric simulation. There is no bloom, cloud system, thumper, true spotlight, imported model, or bitmap material texture; the sampled shadow map stores generated depth.

Slide 09 / Feedback / The player can read and hear the mission

Explain: Feedback makes rescue state observable. The HUD shows secured, aboard, inside, sand, lost, groups, wave, best, pursuit, worm range, altitude, speed, FPS, boost, mute status, action prompts, pickup/unload progress, and breach/extraction timers. Title, pause, and debrief overlays explain the mission and distinguish secured workers from those merely aboard.

Point out: The north-up radar marks the aircraft with an oriented cyan triangle, base with a cyan outline, every running/waiting worker in amber, harvester in white, and worm in red. Text uses an in-code 5-by-7 glyph pattern emitted as colored rectangles, not a font image. Responsive layouts wrap controls and prompts, move or hide secondary panels, and preserve readable text sizes. The HUD streams a dynamic vertex buffer and saves/restores relevant OpenGL state.

Demonstrate: Follow a pickup and delivery counter change, then compare the HUD in different window sizes. Audio has eight spatial sound roles: `ornithopter_flight`, `ornithopter_boost`, `worm_rumble`, `worm_breach`, `harvester_engine`, `rescue_winch`, `rescue_safe`, and `crew_help`. The camera is the listener; inverse-distance attenuation, speed-dependent flight pitch, smoothed boost volume, worm proximity volume, and event-triggered cues reinforce the visuals. Missing files are skipped; engine failure permits silent operation. M or pause reduces master volume to zero, not playback suspension.

Implementation: `arrakis_hud.h` and `arrakis_audio.h` implement these systems. Focus loss clears held inputs and pauses flying; resize recenters steering; fullscreen is switchable; minimized windows wait for events. Frame dt is capped at .1 seconds and mission updates subdivide to at most 1/120 second. CMake builds C++17 with OpenGL, GLFW, GLAD, GLM, and miniaudio, copies assets/audio beside the executable, and registers `RescueMissionLogic` in CTest. The debugger uses the executable directory. The LaTeX report and source references document the pipeline.

Accuracy: Fresh verification of the current build passed 23 of 27 logic cases across 1,057 checks, with five failed assertions in four cases. All four smoke modes passed 120 frames each. Do not claim all tests pass: the changed timer violates older expectations. Render smokes establish limited execution coverage, not complete gameplay, subjective graphics quality, or audible output on every machine.

Slide 10 / Thank You

Explain: Close with Thank You and the three demonstrated requirements: recognizable procedural objects and curved geometry, coordinated mathematical motion and changing scenes, and controllable interaction supported by rendering, feedback, and audio.

Point out: Slides 3-5 provide object and equation evidence; Slides 6-7 provide motion and control evidence; Slides 8-9 provide additional features and honest evaluation. The project report is available alongside the presentation. The already available external demonstration video can be played after this closing slide; no video is embedded in the deck.

Demonstrate: Summarize the complete rescue loop once: find amber crew, board within eight-seat capacity, deliver to the cyan base, and avoid the worm. Use the external recording for sustained motion rather than claiming that a static screenshot proves player input.

Implementation: The outcome combines original generated models, shader-generated surface detail, authored animation, mission state, and an OpenGL frame pipeline. Technical references are in the existing LaTeX report and project source.

Accuracy: Acknowledge opaque glass, constant decorative tanks, GPU-only crater displacement without CPU terrain changes, and the current constant 150-second timer with failing legacy logic assertions. There is no interior cockpit, general physics/navigation engine, bloom, clouds, thumper, or spotlight. The assignment's undefined Karpov objects term is unsupported and is not claimed as an implemented feature.